



Συστήματα Ανάκτησης Πληροφοριών

Εργασία Φάση 1 2024-2025

Κεχριώτη Ελένη

3210078

Ισουφάι Ντέιβιντ

3210056

Εισαγωγή

Στην παρούσα εργασία υλοποιείται και αξιολογείται ένα σύστημα Ανάκτησης Πληροφορίας, με στόχο την αποτελεσματική αναζήτηση επιστημονικών εγγράφων. Το σύστημα βασίζεται στο **Elasticsearch**, μια ισχυρή μηχανή αναζήτησης ανοικτού κώδικα, και εφαρμόζεται πάνω στο σύνολο δεδομένων **SciDocs**, το οποίο περιλαμβάνει μεταδεδομένα, τίτλους και κείμενα επιστημονικών άρθρων. Η επικοινωνία με την Elastic Search γίνεται μέσω python client και της βιβλιοθήκης elasticsearch.

Κατά την ανάπτυξη του συστήματος, δοκιμάστηκαν διαφορετικές τεχνικές αναζήτησης, όπως **match queries**, **dis_max queries**, χρήση **fuzziness**, καθώς και προσαρμοσμένοι **αναλυτές (custom analyzers)** με stopwords και stemming για τη βελτίωση της ακρίβειας. Η αξιολόγηση πραγματοποιήθηκε με βάση το εργαλείο trec_eval, χρησιμοποιώντας μετρικές όπως MAP και avgPre@k, για να εκτιμηθεί η απόδοση του συστήματος ως προς την ανάκτηση σχετικών εγγράφων για κάθε ερώτημα.

Για την εργασία θα πρέπει να κατεβάσετε τις βιβλιοθήκες elasticsearch και nltk.

```
> pip install nltk elasticsearch
```

Στόχοι

Οι βασικοί στόχοι της εργασίας είναι:

- Η υλοποίηση ενός ευέλικτου και αποδοτικού συστήματος αναζήτησης για επιστημονικά κείμενα.
- Η πειραματική αξιολόγηση διαφόρων τεχνικών και παραμέτρων αναζήτησης, όπως ενισχύσεις (boosting), stopwords, stemming, και διαφορετικά query models.
- Η βελτιστοποίηση της απόδοσης ως προς τις μετρικές ανάκτησης (όπως MAP και avgPre@k), μέσα από συστηματική ανάλυση των αποτελεσμάτων.

Μεθοδολογία

Η μεθοδολογία της εργασίας ακολουθεί τα παρακάτω βασικά βήματα:

1. **Προεπεξεργασία δεδομένων:** Τα αρχεία του SciDocs dataset διαμορφώθηκαν σε κατάλληλη μορφή για index στο Elasticsearch, με πεδία όπως title, text, authors, year, και references.

```
def preprocess_text(text):  
  
    text = text.lower()  
  
    text = text.translate(str.maketrans("", "", string.punctuation))  
  
    stop_words = set(stopwords.words("english"))  
  
    words = text.split()  
  
    filtered_words = [word for word in words if word not in stop_words]  
  
    text = " ".join(filtered_words)  
  
    stemmer = PorterStemmer()  
  
    lemmatizer = WordNetLemmatizer()  
  
    word_tokens = word_tokenize(text)  
  
    word_lemma = [lemmatizer.lemmatize(word) for word in word_tokens]  
  
    word_stem = [stemmer.stem(word) for word in word_tokens]  
  
    return {'text':word_tokens, 'word_lemma': word_lemma, 'word_stem':  
word_stem}
```

2. **Δημιουργία ευρετηρίου:** Αρχικά χρησιμοποιήθηκε το προεπιλεγμένο mapping του Elasticsearch. Στη συνέχεια ορίστηκαν custom analyzers με stopwords φίλτρα και stemming, ενώ έγινε πειραματισμός με διαφορετικά similarities.

Αρχικά δοκιμάσαμε ένα απλό ευρετήριο με custom similarity το TF-IDF όπως φαίνεται παρακάτω

```
"similarity": {  
  "scripted_tfidf": {  
    "type": "scripted",  
    "script": {
```

```

        "source": "double tf = Math.sqrt(doc.freq); double idf =
Math.log((field.docCount+1.0)/(term.docFreq+1.0)) + 1.0; double norm =
1/Math.sqrt(doc.length); return query.boost * tf * idf * norm;"
    }
},

```

Τους default analyzers της elasticsearch

```

"analyzer": {
  "default": {
    "type": "english"
  },
  "default_search": {
    "type": "english"
  }
}

```

και mapping με copy_to όλων των πεδίων σε ένα νέο πεδίο allContent.

```

"allContent": {
  "type": "text",
  "similarity": "scripted_tfidf"
}

```

Ωστόσο, δεν ήταν τόσο αποδοτικό όσο θα θέλαμε και σε συνδυασμό με τα παρακάτω query, το MAP έφτανε μόλις σε 0.065.

Ακολούθησαν αρκετοί πειραματισμοί με την δημιουργία του ευρετηρίου, προσπαθώντας να βελτιώσουμε την ποιότητα της ανάκτησης μέσω της τροποποίησης των αναλυτών, των συναρτήσεων ομοιότητας και της κατάλληλης οριοθέτησης των πεδίων.

Τελικά, καταλήξαμε ότι το καλύτερο αποτέλεσμα είχε μια μίξη όλων των προσπαθειών μας.

Αρχικά, ορίστηκε ένα φίλτρο stopwords με την ονομασία scidocs_stop, το οποίο περιλαμβάνει λέξεις που εμφανίζονται πολύ συχνά σε επιστημονικά άρθρα αλλά δεν προσφέρουν ουσιαστική πληροφορία στην αναζήτηση, όπως "study", "figure", "method", "analysis" κ.ά.

```

"scidocs_stop": {
  "type": "stop",
  "stopwords": [

```

```
"the", "and", "of", "in", "to", "a", "is", "for", "on", "that",  
"with", "as", "by", "an", "at", "be", "this", "from", "or", "are",  
"was", "which", "it", "we", "study", "results", "method", "methods",  
"figure", "table", "data", "analysis", "based", "using", "show",  
"shown", "et", "al", "also", "these", "those", "their", "were",  
"may", "can", "used", "use", "such", "have", "has", "had"  
]  
,
```

Παράλληλα, εφαρμόστηκε stemming μέσω του Porter stemmer (english_stemmer), με σκοπό την ενοποίηση διαφορετικών μορφών της ίδιας λέξης (π.χ. "analyzing", "analyzed", "analysis" → "analyz").

```
"english_stemmer": {  
  "type": "porter_stem",  
  "language": "english"  
}
```

Ο προσαρμοσμένος analyzer (scidocs_analyzer) περιλάμβανε:

- Κανονικοποίηση πεζών χαρακτήρων (lowercase)
- Αφαίρεση των custom stopwords
- Εφαρμογή stemming

```
"analyzer": {  
  "scidocs_analyzer": {  
    "type": "custom",  
    "tokenizer": "standard",  
    "filter": [  
      "lowercase",
```

```

        "scidocs_stop",

        "english_stemmer"

    ]

}

}

```

Αυτός ο analyzer εφαρμόστηκε στα βασικά πεδία του ευρετηρίου: title, text, και στο συνθετικό πεδίο allContent, το οποίο δημιουργήθηκε μέσω της λειτουργίας copy_to και συγκεντρώνει αυτόματα το περιεχόμενο των δύο πρώτων πεδίων. Όλα τα παραπάνω πεδία αξιοποιούν τη συνάρτηση ομοιότητας **BM25**.

3. Κατασκευή queries: Πραγματοποιήθηκαν δοκιμές με:

- απλά match queries σε διαφορετικά πεδία,
- dis_max queries για συνδυασμό πηγών πληροφορίας,
- boosting συγκεκριμένων πεδίων,
- χρήση fuzziness για διαχείριση ορθογραφικών αποκλίσεων.

Η αναζήτηση με τα δοσμένα queries έγιναν με διάφορες παραλλαγές των παραμέτρων της μεθόδου search της elasticsearch.

Αρχικά, δοκιμάσαμε απλή αναζήτηση πάνω στο πεδίο all content, όπως φαίνεται παρακάτω.

```

response = client.search(

    index=index_name,

    size=top_k,

    query={

        {"match": {"allContent": query_text}}

    }

)

```

Ωστόσο αυτό παρατηρήσαμε ότι μας έδινε αρκετά χαμηλό MAP score της τάξης του 0.03.

Έπειτα πειραματιστήκαμε με τη χρήση του `dis_max` query και βαρών σε διάφορα πεδία. Μια από τις επιλογές μας ήταν το παρακάτω query, το οποίο επιτρέπει την ανάκτηση εγγράφων που ταιριάζουν καλύτερα σε τουλάχιστον ένα από τα επιμέρους πεδία. Το πεδίο `tie_breaker` ρυθμίστηκε στο 0.3, ώστε τα έγγραφα που ταιριάζουν σε περισσότερα από ένα πεδία να αποκτούν ελαφρώς υψηλότερη συνολική βαθμολογία. Τα `boosts` δόθηκαν εσκεμμένα ώστε το `text` να έχει τον μεγαλύτερο συντελεστή, αφού περιλαμβάνει το κυρίως περιεχόμενο του άρθρου, ενώ το `title` θεωρήθηκε δευτερεύουσας σημασίας αλλά πιο "περιεκτικό" ως προς το νόημα. Η επιλογή αυτών των παραμέτρων προέκυψε εμπειρικά μετά από δοκιμές. Το MAP score εδώ αυξήθηκε αρκετά και έφτασε κοντά στο 0.08.

```
response = client.search(  
    index=index_name,  
    size=top_k,  
    query={  
        "dis_max": {  
            "queries": [  
                {"match": { "title": {"query": query_text, "boost": 3}}},  
                {"match": { "text": {"query": query_text, "boost": 6}}},  
                {"match": { "allContent": {"query": query_text, "boost": 1}}}  
            ],  
            "tie_breaker": 0.3  
        }  
    }  
)
```

Ωστόσο, καταλήξαμε πως το καλύτερο από όλους τους πειραματισμούς μας ήταν το παρακάτω query:

```
response = client.search(  
    index=index_name,
```

```

index=index_name,

size=top_k,

query={

    "bool": {

        "should": [

            {"match": { "title": {"query": query_text,"boost": 1}}},

            {"match": {"text": {"query": query_text,"boost": 1}}},

            {"match": {"allContent": {"query": query_text,"boost":

1}}}]

        ]

    }

}

)

```

Επιλέξαμε τη χρήση του bool query με should όρους ώστε να επιτρέπεται στο Elasticsearch να αξιολογεί έγγραφα που ταιριάζουν έστω και σε ένα από τα πεδία, ενισχύοντας την κάλυψη των αποτελεσμάτων χωρίς να αποκλείονται χρήσιμες περιπτώσεις. Δοκιμάζοντας διαφορετικές τιμές boost, διαπιστώσαμε ότι η εξισορρόπησή τους (ίσα βάρη) παρείχε πιο σταθερά και αξιόπιστα αποτελέσματα σε σχέση με πιο επιθετικές ενισχύσεις, αποφεύγοντας την υπερεστίαση σε συγκεκριμένα πεδία όπως ο τίτλος ή το σώμα κειμένου.

Αυτή η απλή αλλά αποτελεσματική διαμόρφωση αποδείχθηκε ιδιαίτερα αποδοτική για το περιεχόμενο της συλλογής SciDocs.

4. **Αξιολόγηση:** Η αξιολόγηση έγινε με βάση το αρχείο qrels/test.tsv του SciDocs και το εργαλείο trec_eval. Υπολογίστηκαν οι μετρικές:

- **Mean Average Precision (MAP)**
- **avgPre@k**

Για να μπορέσουμε να χρησιμοποιήσουμε το trec_eval χρειάστηκε και να τροποποιήσουμε το αρχείο test.tsv για να είναι σε μορφή που καταλαβαίνει το εργαλείο.

Το test.tsv ήταν στην μορφή

```
query-id      corpus-id      score
```

και τροποποιήθηκε στην παρακάτω μορφή, ενώ του αφαιρέθηκε και η επικεφαλίδα.

```
query-id      0      corpus-id      score
```

```
with open("scidocs/qrels/test.tsv") as f_in, open("trec_eval/test_trec.tsv", "w") as f_out:
    next(f_in)
    for line in f_in:
        parts = line.strip().split("\t")
        query_id, doc_id, score = parts
        f_out.write(f"{query_id} 0 {doc_id} {score}\n")
```

Συμπεράσματα

Η αξιολόγηση των διαφορετικών παραμετροποιήσεων έδειξε ότι η χρήση bool queries με κατάλληλο **boosting στα πεδία text και title**, καθώς και η εφαρμογή **custom analyzers με αγγλικά stopwords και stemming**, οδηγεί σε βελτίωση της απόδοσης. Συγκεκριμένα:

- Το baseline σύστημα με απλά match queries είχε $MAP \approx 0.03$.
- Με dis_max και boost (text:6, title:3), η MAP έφτασε περίπου το 0.08.
- Η χρήση fuzziness δεν βελτίωσε την απόδοση, πιθανώς λόγω της φύσης του corpus (επιστημονικοί όροι με σταθερή γραφή).
- Η βελτιστοποίηση του mapping και του analyzer ήταν πιο καθοριστική από το ίδιο το query structure.

Αποτελέσματα

Τα αποτελέσματα για τα τοπ 20,30,50 του MAP καθώς και η μέση ακρίβεια για τα πρώτα 5,10,15,20 φαίνεται παρακάτω.

	20	30	50
map	0.1019	0.1051	0.1078
avgPre@5	0.1160	0.1160	0.1160
avgPre@10	0.0826	0.0826	0.0826
avgPre@15	0.0649	0.0649	0.0649
avgPre@20	0.0541	0.0541	0.0541

Παρατηρείται μια σταθερότητα στις τιμές των avgPre@k, που φαίνεται κάπως ανησυχητική. Ωστόσο, δεν είναι καθόλου καθώς αξιολογούμε πάντα τα πρώτα k μόνο. Αν ανακτάμε 50 αποτελέσματα αλλά μετράμε την απόδοση στα πρώτα 5, 10, 15 ή 20, το πλήθος των συνολικών εγγράφων που ανακτήθηκαν (20, 30, 50) δεν επηρεάζει το precision στα πρώτα αυτά k, εκτός κι αν αλλάζει η σειρά των εγγράφων. Εδώ όμως δεν αλλάζει, δηλαδή το σύστημα φέρνει τα ίδια πρώτα 20 έγγραφα, γιατί το σύστημα αναζήτησης έχει σταθερή συνάρτηση κατάταξης, την BM25 και σταθερό query με σταθερά βάρη. Αυτό που αλλάζει ελαφρώς είναι το MAP score, αλλά και αυτό είναι φυσιολογικό καθώς λαμβάνει υπόψιν όλα τα ανακτηθέντα κείμενα.